



Prog-Py

FS2026 – Prof. Selina Rea Malacarne

Autoren: Mirko Ratti

<https://github.com/mirkoratti/FS2026>

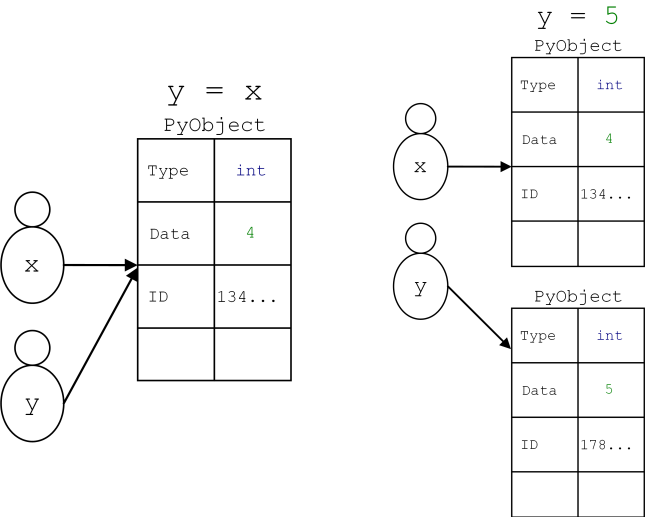
V1.0.20260605

1 Python Overview

- **Ursprung:** Entwickelt von Guido van Rossum mit Fokus auf Einfachheit und Stärke.
- **Standardbib.:** Bekannt für "Batteries included", also eine umfangreiche, integrierte Bibliothek.
- **Ökosystem:** Für Ingenieure ersetzt Python Matlab-Toolboxen durch Pakete wie **NumPy** (Zahlen), **pandas** (Daten) und **matplotlib** (Grafik).

2 Variablen und Objekte

In Python ist alles ein Objekt. Variablen sind keine Container, sondern an Objekte gebundene **Referenzen** (Pointer).



2.1 Integrierte Datentypen

Kategorie	Datentyp	Veränderbarkeit	Falsch-Wert
None	NoneType	Unveränderlich	None
Numerisch	int	Unveränderlich	0
	float	Unveränderlich	0.0
	bool	Unveränderlich	False
	complex	Unveränderlich	0 + 0j
Sequenz	str	Unveränderlich	"
	list	Veränderlich	[]
	tuple	Unveränderlich	()
	bytes	Unveränderlich	b"
	bytearray	Veränderlich	bytearray(b"
Mengen	set	Veränderlich	set()
	frozenset	Unveränderlich	frozenset()
Assoziativ	dict	Veränderlich	{}

2.2 Veränderbarkeit

- **Unveränderlich:** Objekte, die nach Erstellung nicht modifiziert werden können (z.B. int, float, str, tuple). Wertänderung erzeugt real ein komplett neues Objekt im Speicher.
- **Veränderlich:** Objekte, die direkt in-place modifiziert werden können (z.B. list, set, dict).

Die Speicheradresse eines Objekts kann mit der Funktion `id()` visualisiert werden. Sie gibt eine eindeutige Ganzzahl für das jeweilige Objekt zurück.

3 Namensräume und Gültigkeitsbereich

Ein Namensraum ist ein System (Dictionary), das Eindeutigkeit und konfliktfreie Nutzung aller Namen im Programm sichert.

- **Built-in:** Enthält alle in Python verfügbaren, integrierten Symbole (z.B. `print()`).
- **Global:** Symbole, definiert auf der obersten Ebene eines Moduls oder Skripts.
- **Lokal:** Erstellt für spezifische Bereiche, wie etwa innerhalb einer Funktion.
- **LEGB-Regel:** Python sucht Namen von innen nach aussen: Lokal → Global → Built-in.

Built-in Symbole können überschrieben werden, was jedoch absolut nicht empfohlen wird.

4 Arithmetik und Magic Methods

- **Spezielle Operatoren:** `//` (Ganzzahldivision), `%` (Modulo/Rest) und `**` (Potenzierung).
- **Syntactic Sugar:** Operatoren sind ein benutzerfreundlicher Weg zum Aufruf interner Methoden (z.B. wird `x + y` intern als `x.__add__(y)` ausgeführt).

Operator	Dunder Methode	Beschreibung
<code>x + y</code>	<code>x.__add__(y)</code>	Summe
<code>x - y</code>	<code>x.__sub__(y)</code>	Differenz
<code>x * y</code>	<code>x.__mul__(y)</code>	Produkt
<code>x / y</code>	<code>x.__truediv__(y)</code>	Quotient
<code>x // y</code>	<code>x.__floordiv__(y)</code>	Ganzzahl-Quot. ¹
<code>x % y</code>	<code>x.__mod__(y)</code>	Rest (Modulo) ¹
<code>x ** y</code>	<code>x.__pow__(y)</code>	Potenzierung (x^y)
<code>abs(x)</code>	<code>x.__abs__()</code>	Absolutwert
<code>-x</code>	<code>x.__neg__()</code>	Negatives Vorzeichen

¹ Operator für Datentyp complex nicht definiert

Code-Beispiel

So interpretiert Python die Operatoren hinter den Kulissen:

```
1 # Standard-Operatornutzung
2 result_a = 5 + 5
3
4 # Hinter den Kulissen (Magic Method)
5 result_b = (5).__add__(5)
6
7 print(result_a == result_b) # Ausgabe: True
```

5 Bedingte Verzweigung

- **Basisstrukturen:** `if`, `elif` und `else` führen Code bedingungs-basiert aus. Konsistente Einrückung ist zwingend.
- **Match-case:** Eingeführt in Python 3.10 zur Erkennung spezifischer Muster und Datenstrukturen.

```
1 x = 5
2
3 if x == 5:
```

```
4     print("5")
5 else:
6     print("Not 5")
7
8 match x:
9     case 5:
10        print("5")
11     case 4:
12        print("4")
13     case _:
14        print("Default case")
```

5.1 Boolesche Ausdrücke und Operatoren

- **Walrus-Operator (`:=`):** Weist einen Wert zu und gibt ihn direkt innerhalb eines Ausdrucks zurück.
- **Ternärer Operator:** Kompakte Syntax für einfache, einzeilige bedingte Zuweisungen.
- **`==` vs `is`:**
 - `==` prüft auf reine Wertgleichheit;
 - `is` prüft Objektidentität im Speicher (ideal für `None`-Prüfung).
- **in-Operator:** Prüft Vorhandensein in Objekten via `__contains__()` (Nur für Iterables nutzbar: Listen, Strings, Dicts).

```
1 # Walrus Operator (:=)
2 if (a := int(input("Enter a number: "))) > 3:
3     print("Number is greater than 3.")
4
5 # Ternary Operator
6 b = 5
7 a = b if b > 3 else 0
8
9 # == vs is
10 my_list = [1, 2, 3]
11 print(my_list == [1, 2, 3]) # True (values are identical)
12 print(my_list is [1, 2, 3]) # False (different objects in memory)
13
14 # in Operator
15 fruits = ["banana", "apple"]
16 print("apple" in fruits) # True
```

5.2 Ausnahmebehandlung (Exception Handling)

- **LBYL (Look Before You Leap):** Defensiver Stil, prüft explizit Bedingungen vor dem Aufruf. Anfällig für "Race Conditions" (Zustand ändert sich zw. Prüfung und Aktion).
- **EAFP (Easier to Ask Forgiveness...):** Pythonischer Ansatz. Geht von Erfolg aus und nutzt `try/except`. Meist sauberer und robuster in Multi-Threading.
- **Exceptions fangen:** Immer spezifische Exceptions fangen, keine generischen. `as-` Keyword zur Inspektion des Exception-Objekts verwenden.
- **Ablaufsteuerung:**
 - `else:` Läuft nur, wenn der `try`-Block fehlerfrei erfolgreich war.
 - `finally:` Läuft immer (z.B. für Dateifreigabe), ungeachtet dessen, ob eine Exception auftrat.

```
1 # EAFP: try, except, else, finally
2 try:
3     x = int(input("Enter a number other than zero: "))
4     y = 1 / x
5 except ValueError:
```

```
6     print('Oops. Not a valid integer.')
```

```
7 except ZeroDivisionError as e:
```

```
8     print('Oops! ' + str(e))
```

```
9 else:
```

```
10    print(f'Success! Result: {y}')
```

```
11 finally:
```

```
12    print("Always executed.")
```

```
13
```

```
14 # LBYL
```

```
15 inp = input("Enter a number other than zero: ")
```

```
16
```

```
17 if(inp.isdecimal()):
```

```
18     x = int(inp)
```

```
19     if x != 0:
```

```
20         print(1/x)
```

```
21     else:
```

```
22         print("Wrong Input!")
```

```
23 else:
```

```
24     print("Wrong Input!")
```

6 Iterationen

6.1 Slicing

- **Syntax:** part = x[start:end:step].
- Start ist inklusiv (Standard 0).
- End ist exklusiv (Standard End-Index).
- Step ist optional (Standard 1). Negativer Step bedeutet umgekehrte Reihenfolge.

```
1 data = [0, 1, 2, 3, 4, 5.0, [1,2,3]]
```

```
2 part = data[::-1] # Umgekehrte Reihenfolge
```

6.2 For-Schleifen

- Ruft intern iter() auf, um einen Iterator zurückzugeben.
- Nutzt die next()-Methode fürs nächste Element, bis die StopIteration-Exception auftritt.

```
1 for value in iterable:
```

```
2     expression1
```

6.2.1 Schleifensteuerung

- **break:** Beendet eine Schleife vor ihrer vollständigen Abarbeitung.
- **continue:** Überspringt den restlichen Code der Schleife in der aktuellen Iteration.
- **else-Keyword:** Wird ausgeführt, wenn die Schleife normal ohne ein break endet.

6.2.2 Dictionaries

```
1 for key in data: # Iteriert Schluessel
```

```
2 for value in data.values(): # Iteriert Werte
```

```
3 for key, value in data.items(): # Iteriert Schluessel und
```

```
    Werte
```

6.2.3 Enumerate

- Wird benutzt, wenn beim Iterieren über Elemente ein Zähler benötigt wird.

```
1 for i, fruit in enumerate(fruits, start=10):
```

```
2     print(i, fruit)
```

6.2.4 Zip

- Iteriert über mehrere Iterables gleichzeitig.
- **Objekte verschiedener Längen:** zip() stoppt, sobald das kürzeste Iterable aufgebraucht ist. Restliche Elemente werden ignoriert.

```
1 colors = ["yellow", "red", "yellow", "blue", "green"] # Length
```

```
5
```

```
2 fruits = ["banana", "apple", "lemon", "blueberry"] # Length
```

```
4
```

```
3
```

```
4 # The loop will only run 4 times
```

```
5 for color, fruit in zip(colors, fruits):
```

```
6     print(color, fruit)
```

6.3 Comprehensions

- **Basis-Syntax:** [expression for item in iterable].
- **Verschachtelte for-Schleifen:** Von links nach rechts ausgewertet.
- **Funktionsaufrufe:** Built-in Funktionen wie any() können benutzt werden.
- **Bedingte Syntax:** Die Position der if-Anweisung ändert das Verhalten komplett.
 - **Filtern (Nur if):** Muss ans **ENDE** gesetzt werden. Filtert Elemente aus dem Iterable heraus.
 - **Modifizieren (if/else):** Muss an den **ANFANG** (vor for). Ändert den Elementwert bedingungs basiert, filtert aber nicht.

```
1 # 1. Nested loops
```

```
2 # Syntax: [expression for outer_item in outer_iterable
```

```
    inner_item in inner_iterable]
```

```
3 fields = [b + str(n) for b in letters for n in numbers]
```

```
4
```

```
5 # Normal form equivalent
```

```
6 for b in letters:
```

```
7     for n in numbers:
```

```
8         b + str(n)
```

```
9
```

```
10 # 2. ONLY IF (Filtering) -> Placed at the END
```

```
11 # Syntax: [expression for item in iterable if condition]
```

```
12 filtered_values = [item**2 for item in range(1, 11) if (item
```

```
    **2) % 4 != 0]
```

```
13
```

```
14 # 3. IF/ELSE (Modifying) -> Placed at the BEGINNING
```

```
15 # Syntax: [expression_if_true if condition else
```

```
    expression_if_false for item in iterable]
```

```
16 modified_values = [item**2 if (item**2) % 4 != 0 else 0 for
```

```
    item in range(1, 11)]
```

7 Integrierte Methoden

7.1 Listen

Methoden sort() und sorted()

- sort(): Modifiziert die Liste an Ort und Stelle. Ermöglicht einen reverse=True-Parameter für absteigende Reihenfolge.
- sorted(): Gibt eine neue sortierte Liste zurück und kann auf alle iterierbaren Objekte angewendet werden.
- **Das key-Argument:** Ermöglicht das Festlegen einer benutzerdefinierten Sortierfunktion. Es akzeptiert eingebaute Funktionen (z.B. str) oder anonyme lambda-Funktionen für inline Sortierkriterien.
- **Algorithmusstabilität und mehrere Prioritäten:** Python nutzt *Timsort*, einen stabilen Sortieralgorithmus. Stabilität bedeutet, dass, wenn zwei Elemente denselben Sortierschlüssel haben, sie ihre ursprüngliche relative Reihenfolge beibehalten. Um nach mehreren Kriterien zu sortieren, können Sie 'aufeinanderfolgende

Sortierungen stapeln': Sie müssen die *geringste* Prioritäts-Sortierung zuerst anwenden und den *Haupt*-Prioritätsfilter zuletzt. Alternativ können Sie eine einzelne Lambda-Funktion verwenden, die ein Tupel zurückgibt, das die Hierarchie der Prioritäten darstellt.

```
1 data_list = [(1, "D"), (3, "B"), (2, "B"), (5, "E"), (4, "A"),
```

```
    (6, "D")]
```

```
2
```

```
3 # Approach 1: Stacking sorts (exploiting Timsort stability)
```

```
4 # The least priority sort goes first
```

```
5 data_list.sort(key=lambda x: str(x[0]))
```

```
6 # The main priority filter goes last
```

```
7 data_list.sort(key=lambda x: str(x[1]))
```

```
8
```

```
9 # Approach 2: Single sort with a tuple hierarchy
```

```
10 # Equivalent to the stacked approach above
```

```
11 data_list.sort(key=lambda x: (str(x[1]), str(x[0])))
```

Indirektes Sortieren (Sortierindizes) In vielen Algorithmen ist es sehr effizient, ein Array von Indizes zu sortieren, statt die eigentlichen Datenelemente hin- und herzubewegen. Indem wir eine Lambda-Funktion verwenden, können wir eine Standardsequenz von Indizes sortieren, basierend auf den Werten, auf die sie in der ursprünglichen Liste zeigen.

```
1 original = ["F", "B", "C", "G", "A", "D", "E"]
```

```
2 # Generate an array of indices: [0, 1, 2, 3, 4, 5, 6]
```

```
3 idxs = list(range(len(original)))
```

```
4
```

```
5 # Sort the indices based on the alphabetical value of the
```

```
    original elements
```

```
6 sorted_idx = sorted(idxs, key=lambda i: original[i])
```

```
7 # Result: [4, 1, 2, 5, 6, 0, 3] (e.g., index 4 points to "A")
```

```
8
```

```
9 # Reconstruct the sorted array using the sorted indices
```

```
10 result = [original[i] for i in sorted_idx]
```

```
11
```

```
12 # Generate a reverse mapping to find where original elements
```

```
    ended up
```

```
13 reverse_idx = [result.index(char) for char in original]
```

Listen-Slicing Beim Slicing werden Teilsequenzen mit der Syntax x[start:end:step] extrahiert. Neben der Extraktion ist Slicing äußerst leistungsfähig, um Listen dynamisch zu verändern: Es ermöglicht, einen gesamten Bereich von Elementen zu ersetzen oder mehrere Elemente an einem bestimmten Index gleichzeitig einzufügen, ohne vorhandene Daten zu überschreiben.

```
1 data = ['a', 'b', 'c']
```

```
2 # Mehrere Elemente einfügen, ohne zu ersetzen (bei Index 3)
```

```
3 data[3:3] = ['#', '$'] # Ergebnis: ['a', 'b', 'c', '#', '$']
```

```
4
```

```
5 # Einen gesamten Bereich durch neue Elemente ersetzen
```

```
6 data[1:3] = ['D', 'E'] # Ergebnis: ['a', 'D', 'E', '#', '$']
```

7.2 Shallow und Deep Copy

Die Zuweisung einer Variable bindet lediglich einen Namen an ein Objekt; sie erstellt keine Kopie.

- **Shallow Copy:** Funktionen wie list(), dict() oder .copy() erstellen ein neues Objekt nur auf der obersten Ebene. Untergeordnete verschachtelte Objekte werden lediglich referenziert, d.h. Änderungen an veränderbaren verschachtelten Objekten wirken sich sowohl auf das Original als auch auf die Kopie aus.

- **Deep Copy:** Erstellt eine echte, rekursive Kopie des Objekts und all seiner Unterobjekte.

```
1 import copy
2 list1 = [[1, 2], [3, 4]]
3 list_shallow = list1.copy()
4 list_deep = copy.deepcopy(list1)
```

7.3 Strings

Meistgenutzte Methoden

- **Aufteilen und Bereinigen:** `split(sep)` teilt anhand eines Trennzeichens. `splitlines()` teilt an Zeilenumbrüchen. `strip()`, `lstrip()` entfernen führende/trailing Zeichen.
- **Suchen und Ersetzen:** `find(sub).count(sub).replace(old, new)`.
- **Schreibweise:** `lower().upper().capitalize().title()`.
- **Testen:** `isdigit().isalpha().isalnum().islower().isupper()`.

join() und Generatoren Die Methode `join()` kombiniert alle String-Elemente eines iterierbaren Objekts zu einem einzigen String und verwendet den aufrufenden String als Trennzeichen. Sie ist besonders effizient, wenn sie mit Generator-Ausdrücken kombiniert wird.

```
1 z = [(10, "Matthias"), (20, "Annabelle")]
2 # Elemente mithilfe eines Generator-Ausdrucks verbinden
3 ranking = "\n".join(f"{name} ({points} Points)" for points,
4                       name in z)
```

F-Strings Formatierung Die Formatierungslogik folgt der Syntax: `{<f_expression> [->] [!<conversion>] [<format_spec>]}`. Selbstdokumentierende Ausdrücke können erstellt werden, indem man ein Gleichheitszeichen = nach dem Variablennamen hinzufügt.

Parameter	Beschreibung	Beispiel	Ausgabe
Darstellungstyp (<type>)			
b	Binäre Ganzzahl	f'{10:b}'	'1010'
c	Einzelzeichen	f'{65:c}'	'A'
d	Dezimale Ganzzahl	f'{75:d}'	'75'
e oder E	Exponentielle Darstellung	f'{0.001:e}'	'1.000000e-03'
f oder F	Gleitkommazahl	f'{157:f}'	'157.000000'
g oder G	Gleitkommazahl oder Exponential (abhängig von der Größe)	f'{15723823:G}'	'1.57238E+07'
o	Oktale Ganzzahl	f'{16:o}'	'20'
s	Zeichenkette	f'{"Hallo:s}"	'Hallo'
x oder X	Hexadezimale Ganzzahl	f'{10:X}'	'A'
%	Prozent	f'{1:%}'	'100.000000%'
Ausrichtung (<align>)			
<	Links ausgerichtet	f'{12:<5}'	'12 '
>	Rechts ausgerichtet	f'{12:>5}'	'12 '
^	Zentriert	f'{12:^5}'	'12 '
=	Vorzeichen wird ganz links erzwungen	f'{-12:=5}'	'- 12'
Füllzeichen (<fill>)			
char	Beliebiges Zeichen (verwendet mit align)	f'{12:>5}'	'__12'
Vorzeichen (<sign>)			
+	Erzwingt Vorzeichen für positive/negative	f'{123:+}'	'+123'
-	Vorzeichen nur für negative Zahlen	f'{-123:-}'	'-123'
Gruppierung (<group>)			
, oder _	Tausendertrennzeichen	f'{1000:,}'	'1,000'
Breite und Genauigkeit (<width>.<precision>)			
width	Minimale Ausgabebreite	f'{12:5}'	'12 '
.precision	Dezimalstellen (für Gleitkommazahlen)	f'{3.14159:.2f}'	'3.14'

Mit # ist es möglich, das alternative Ausgabeformat anzuzeigen (0xA10, 0b01101)

7.4 Dictionaries

Meistgenutzte Methoden

- **get(key, default):** Gibt den Wert für einen Schlüssel zurück oder einen Standardwert (wie None), falls der Schlüssel nicht existiert.
- **keys(), values(), items():** Geben jeweils Ansichten der Schlüssel, Werte und Schlüssel-Wert-Paare des Dictionaries zurück.
- **setdefault(key, default):** Gibt den Wert eines Schlüssels zurück; falls er fehlt, fügt es den Schlüssel mit dem angegebenen Standardwert ein.

- **pop(key)** und **popitem():** Entfernt ein bestimmtes Schlüssel-Wert-Paar oder das zuletzt hinzugefügte Paar.
- **clear():** Entfernt alle Schlüssel-Wert-Paare.

7.5 Dateiverwaltung

Dateien werden mithilfe der Funktion `open()` behandelt, die einen Dateinamen und einen Modus akzeptiert (z.B. 'r' zum Lesen, 'w' zum Schreiben, 'a' zum Anhängen). Das Festlegen von `encoding='utf-8'` wird dringend empfohlen, um Probleme bei der Zeichenauslegung zwischen Plattformen zu vermeiden.

Die `with`-Anweisung stellt einen Python-typischen Kontextmanager bereit, der sicherstellt, dass die Datei beim Verlassen des Blocks automatisch geschlossen wird. Das Modul `pathlib` ermöglicht robuste, objektorientierte Pfadmanipulationen mithilfe des `/`-Operators und stellt so eine nahtlose Ausführung in Umgebungen unter Unix/macOS sicher.

```
1 import pathlib
2
3 # Composing a path object utilizing the / operator
4 file_path = pathlib.Path.cwd() / 'recipes' / "output.txt"
5
6 # Writing files safely using the 'with' context manager
7 with open(file_path, "w", encoding="utf-8") as f:
8     f.write("String data\n")
9     f.writelines(["Line 2\n", "Line 3\n"])
10
11 # Reading files line by line directly via the file object
12 with open(file_path, "r", encoding="utf-8") as f:
13     for line in f:
14         print(line.strip())
```

8 Erweitertes Unpacking und Funktionen

8.1 Tuple Packing und Unpacking

Python ermöglicht das Gruppieren und Verteilen von Objekten über Mechanismen zum Packen und Entpacken.

- **Packing:** Mehrere einzelne Objekte werden zu einem einzigen Tupel gruppiert.
- **Unpacking:** Elemente in einem iterierbaren Objekt werden einzelnen Variablen zugewiesen. Dafür muss die Anzahl der Variablen der Anzahl der Elemente entsprechen, sofern kein erweitertes Entpacken verwendet wird.

```
1 # Packing
2 employee = 'Peter', 'Muller' # Ergibt ('Peter', 'Muller')
3
4 # Unpacking
5 first, last = employee
6
7 # Mehrfachzuweisung (kombiniertes Packing/Unpacking)
8 x, y, z = 5, 10, 15
```

8.1.1 Erweitertes Iterable-Unpacking

Das Entpacken funktioniert mit jedem iterierbaren Objekt (Strings, Listen, Mengen) und unterstützt bestimmte Operatoren, um variable Längen oder unerwünschte Werte zu behandeln.

- **Rest-Operator (*):** Erfasst die verbleibenden Elemente in einer Liste.
- **Platzhalter (_):** Wird konventionell verwendet, um bestimmte Werte beim Entpacken zu ignorieren.

```
1 # Unpacking mit Rest
2 first, last, *address, birth_date = ("Peter", "Muller", "Street", 10, "14.04.1977")
```

```
3 # address wird ["Street", 10]
4
5 # Werte ignorieren
6 first, last, _, birth_date = employee_data
7
8 # Entpacken in verschachtelten Strukturen
9 x = [1, 2, (3, [4, 5])]
10 a, b, (c, (d, e)) = x
```

8.1.2 Iterable-Expansion

Die Operatoren `*` und `**` ermöglichen das Entpacken iterierbarer Objekte in anderen Collection-Literalen.

```
1 list1 = [1, 2]
2 list2 = [*list1, 3, 4] # [1, 2, 3, 4]
3
4 dict1 = {"A": 1, "B": 2}
5 dict2 = {"C": 3, **dict1} # {'C': 3, 'A': 1, 'B': 2}
```

8.2 Funktionen

Funktionen werden mit dem Schlüsselwort `def` definiert. Sie legen einen lokalen Gültigkeitsbereich fest und können Objekte an den Aufrufer zurückgeben.

- **Parameter:** Platzhalter, die in der Funktionssignatur definiert sind.
- **Argumente:** Tatsächliche Werte, die beim Aufruf übergeben werden.
- **Rückgabewert:** Funktionen geben standardmäßig None zurück. Mehrere Werte können als Tupel zurückgegeben werden.

8.2.1 Argumenttypen und Einschränkungen

- **Positional vs. Keyword:** Positionsargumente müssen vor Keyword-Argumenten in einem Aufruf stehen.
- **Standardparameter:** Parameter mit Standardwerten müssen auf diejenigen ohne Standardwert folgen.
- **Variadische Argumente (*args, **kwargs):** `*args` sammelt zusätzliche positionbezogene Argumente in ein **tuple**; `**kwargs` sammelt zusätzliche Keyword-Argumente in ein Dictionary.

8.2.2 Argument-Erzwingung

Entwickler können bestimmte Aufrufstile erzwingen, indem sie `/` und `*` verwenden.

```
1 def constrained_func(pos_only, /, pos_or_kw, *, kw_only):
2     pass
3
4 # / : Alle Argumente vor mÃssen positionsspezifisch sein.
5 # * : Alle Argumente nach mÃssen keyword-only sein.
```

8.3 Docstrings

Dokumentationsstrings sollten der Funktionsdefinition folgen, um ihr Verhalten zu beschreiben. Sie können über `help(function_name)` oder das `__doc__`-Attribut abgerufen werden.

```
1 def add(a, b):
2     """Gibt die Summe von a und b zurÃck."""
3     return a + b
```

8.4 Lambda-Funktionen

Lambdas sind anonyme Funktionen mit einem einzigen Ausdruck. Sie werden hauptsächlich als "keyFunktionen zum Sortieren oder Filtern verwendet.

Syntax: `lambda parameters: expression`


```
1 # Sortieren einer Liste von Strings nach dem numerischen Teil
2 articles = ['art1', 'art20', 'art3']
3 sorted_list = sorted(articles, key=lambda x: int(x[3:]))
```

9 Objektorientierte Programmierung (OOP)

Der grundlegende Grundsatz der OOP ist die Kapselung von Daten (Attributen) und Logik (Methoden) in Objekten. Diese Struktur stellt die Datenintegrität sicher, indem sie die direkte externe Manipulation einschränkt.

9.1 Klassendefinition und Instanziierung

Eine Klasse dient als formale Blaupause. Gemäß PEP 8 folgen Klassennamen der Konvention CapWords. Objekte sind konkrete Instanzen, die aus diesen Blaupausen erzeugt werden.

```
1 class Cat:
2     """Formale Struktur einer Katze."""
3     pass
4
5 # Instanziierung
6 garfield = Cat()
```

9.2 Attribute und Zustandsintegrität

Attribute speichern Daten innerhalb eines Objekts. Python unterscheidet zwischen Klassen- und Instanzvariablen anhand ihres Geltungsbereichs und ihrer Definition.

9.2.1 Klassen- vs. Instanzvariablen

- **Klassenvariablen:** Direkt innerhalb des Klassentextes definiert. Sie werden über alle Instanzen hinweg geteilt.
- **Instanzvariablen:** Mit `self` definiert. Sie sind eindeutig für jede Instanz.

Wichtige Warnung: Ändere niemals eine Klassenvariable über eine Instanz (z.B. `instance.class_var = value`). Dadurch wird ein neues Instanzattribut erzeugt, das die Klassenvariable für dieses spezifische Objekt überdeckt. Aktualisiere Klassenvariablen immer über den Klassennamen: `ClassName.class_var = value`.

9.2.2 Initialisierung und dynamische Attribute

Es wird dringend empfohlen, **alle** Instanzvariablen innerhalb der `__init__`-Methode zu definieren.

- **Vermeide 'On-the-fly'-Attribute:** Das macht die Struktur des Objekts unvorhersehbar und erschwert das Debugging.
- **Definierter Zustand:** Sicherstellen, dass alle Attribute im Konstruktor initialisiert werden

9.3 Methoden-Attribute und self

Methoden sind Funktionen, die innerhalb einer Klasse definiert sind. Der erste Parameter ist immer `self`, was die Instanz darstellt, die die Methode aufruft. Python bindet die Instanz automatisch während des Aufrufs an diesen Parameter.

```
1 account1.deposit(100)
2 # Wird im Hintergrund ausgeführt
3 class_name.deposit(self=account1, amount=100)
```

9.4 Datenabstraktion und Kapselung

Kapselung verbirgt Implementierungsdetails und legt nur die notwendigen Informationen über definierte Schnittstellen offen. Python verwendet Namenskonventionen, um die beabsichtigte Sichtbarkeit von Attributen zu signalisieren:

Kategorie	Konvention	Beschreibung
Öffentlich	<code>name</code>	Von überall aus zugänglich.
Geschützt	<code>_name</code>	Unerwünscht, extern darauf zuzugreifen; für Vererbung verwendet.
Privat	<code>__name</code>	Versteckt durch Name Mangling; nicht direkt zugreifbar.

Eine private Variable ist `my_object._MyClass__private_var`

9.5 Properties

Das Property-Objekt wird mit der Signatur erstellt:

`var = property(fget=None, fset=None, fdel=None, doc=None)`

```
1 class BankAccount:
2     def __init__(self, balance):
3         self._balance = balance
4
5     def _get_balance(self):
6         print("Getter called")
7         return self._balance
8
9     def _set_balance(self, value):
10        print("Setter called")
11        self._balance = value
12
13    # Background implementation of the property
14    balance = property(_get_balance, _set_balance)
```

9.5.1 Decorators

Das Gleiche könnte auch mit Decorators erreicht werden:

```
1 class BankAccount:
2     def __init__(self, balance):
3         self._balance = balance
4
5     @property
6     def balance(self):
7         """Getter: returns the balance."""
8         print("Balance requested.")
9         return self._balance
10
11    @balance.setter
12    def balance(self, value):
13        """Setter: updates the balance."""
14        if value >= 0:
15            self._balance = value
16        else:
17            print("Invalid amount.")
```

10 Magic Methods und benutzerdefiniertes Verhalten

10.1 Objektrepräsentation

Magic Methods (oder dunder methods) ermöglichen es benutzerdefinierten Klassen, mit eingebauten Python-Funktionen und Operatoren zu interagieren.

- `__str__(self)`: Gibt eine benutzerfreundliche Zeichenkettenrepräsentation zurück (wird von `print()` verwendet).
- `__repr__(self)`: Gibt eine eindeutige Zeichenkettenrepräsentation zurück, die idealerweise zum Debuggen oder zum erneuten Erstellen des Objekts verwendet wird.

```
1 class Book:
2     def __init__(self, title, author):
3         self.title = title
```

```
4         self.author = author
5
6     def __str__(self):
7         return f'{self.title} by {self.author}'
8
9     def __repr__(self):
10        return f"Book(title='{self.title}', author='{self.author}')"

```

10.2 Emulation von Container-Typen

Klassen können sich wie Sequenzen (Listen) oder Abbildungen (Dictionaries) verhalten, indem sie bestimmte Methoden implementieren:

- `__len__(self)`: Gibt die Anzahl der Elemente zurück.
- `__getitem__(self, key)`: Definiert das Verhalten für Indizierung (`obj[key]`).
- `__setitem__(self, key, value)`: Definiert das Verhalten für die Zuweisung von Elementen (`obj[key] = value`).

```
1 class Inventory:
2     def __init__(self):
3         self._items = {}
4
5     def __setitem__(self, key, value):
6         self._items[key] = value
7
8     def __getitem__(self, key):
9         return self._items.get(key, "Item not found")
10
11    def __len__(self):
12        return len(self._items)
```

10.3 Context Manager

Context Manager stellen sicher, dass Ressourcen (wie Dateien oder Netzwerkverbindungen) mithilfe der `with`-Anweisung korrekt verwaltet werden. Dafür müssen zwei Methoden implementiert werden:

- `__enter__(self)`: Wird am Anfang des `with`-Blocks ausgeführt. Gibt die Ressource zurück.
- `__exit__(self, exc_type, exc_val, tb)`: Wird am Ende ausgeführt, auch wenn eine Ausnahme auftritt.

```
1 class FileManager:
2     def __init__(self, filename, mode):
3         self.filename = filename
4         self.mode = mode
5
6     def __enter__(self):
7         self.file = open(self.filename, self.mode)
8         return self.file
9
10    def __exit__(self, exc_type, exc_val, traceback):
11        self.file.close()
12
13    with FileManager("data.txt", "w") as f:
14        f.write("Structured data")
```

10.4 Reflektierte Arithmetik-Methoden ('r'-Präfix)

Python unterscheidet zwischen einer Operation, bei der sich ein Objekt auf der linken Seite (LHS) befindet, und einer Operation, bei der es sich auf der rechten Seite (RHS) befindet. Das ist entscheidend für die Interoperabilität mit eingebauten Typen (wie `int` oder `float`).

- `__add__(self, other)`: Wird für `self + other` aufgerufen.
- `__radd__(self, other)`: Wird für `other + self` nur dann aufgerufen, wenn das LHS-Objekt `__add__` für den spezifischen Typ von `self` nicht definiert.

```

1 class Price:
2     def __init__(self, amount):
3         self.amount = amount
4
5     def __add__(self, other):
6         # Behandelt: Price(10) + 5
7         val = other.amount if isinstance(other, Price) else
8         other
9         return Price(self.amount + val)
10
11     def __radd__(self, other):
12         # Behandelt: 5 + Price(10)
13         # Verwendet __add__, da Addition kommutativ ist
14         return self.__add__(other)

```

10.5 Das NotImplemented-Singleton

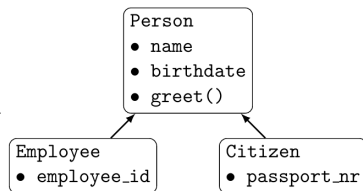
In Python ist `NotImplemented` ein spezielles Objekt (ein Singleton), das von binären Methoden (wie `__add__`, `__sub__`, `__eq__`, usw.) zurückgegeben wird, um anzuzeigen, dass die Operation für den Datentyp des anderen Operanden nicht implementiert ist.

Wichtig: Es darf nicht mit `NotImplementedError` verwechselt werden, das eine Ausnahme ist. `NotImplemented` ist ein Rückgabewert, der Python signalisiert, eine alternative Strategie auszuprobieren.

11 Vererbung und Polymorphie

Vererbung ermöglicht es einer neuen Klasse (Unterklasse oder abgeleiteten Klasse), die Eigenschaften und Methoden einer bestehenden Klasse (Oberklasse oder Basis-klasse) zu übernehmen.

- Code-Wiederverwendung
- Modularität
- und die Etablierung einer “is-a”-Beziehung.



Polymorphie ermöglicht es Unterklassen, Methoden mit demselben Namen wie diejenigen in der Oberklasse zu definieren und sie dadurch effektiv zu überschreiben. Die konkret aufgerufene Methode hängt vom tatsächlichen Objekttyp zur Laufzeit ab und erhöht die Flexibilität der Schnittstelle.

11.0.1 Die super()-Funktion

Die Funktion `super()` stellt den Zugriff auf Methoden und Attribute der Oberklasse bereit. Sie wird hauptsächlich innerhalb der Methode `__init__` der Unterklasse verwendet, um geerbte Attribute zu initialisieren und so sicherzustellen, dass der Basisobjektzustand korrekt eingerichtet wird.

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5 class Employee(Person):
6     def __init__(self, name, employee_id):
7         super().__init__(name) # Initialize superclass
8         attributes
9         self.employee_id = employee_id

```

11.0.2 Abstrakte Klassen

Abstrakte Klassen dienen als Vorlagen und können nicht direkt instanziiert werden. Sie definieren abstrakte Methoden, die von abgeleiteten Klassen implementiert werden müssen, wodurch eine konsistente Schnittstelle erzwungen wird. Dafür ist das `abc`-Modul erforderlich.

```

1 from abc import ABC, abstractmethod
2
3 class Vehicle(ABC):
4     @abstractmethod
5     def accelerate(self):
6         pass
7
8 class Car(Vehicle):
9     def accelerate(self):
10        print('The car accelerates')

```

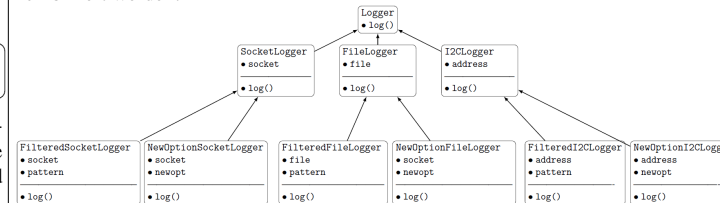
11.1 Designprinzipien (SOLID)

Beim Aufbau von Vererbungshierarchien stellt die Einhaltung von Designprinzipien sicher, dass Flexibilität und Wartbarkeit gewährleistet sind:

- **Single Responsibility:** Eine Klasse sollte nur eine Verantwortung haben und für genau eine Sache verantwortlich sein.
- **Open-Closed:** Klassen sollten für Erweiterungen offen (z. B. über Vererbung) und für Änderungen geschlossen sein.
- **Liskov Substitution:** Objekte einer Oberklasse sollten durch Objekte einer Unterklasse ersetzbar sein, ohne die Korrektheit des Programms zu beeinträchtigen.
- **Interface Segregation:** Clients sollten nicht dazu gezwungen werden, von nicht verwendeten Methoden abhängig zu sein.
- **Dependency Inversion:** Hochrangige Module sollten nicht von niedrigrangigen Modulen abhängen; beide sollten von Abstraktionen abhängen.

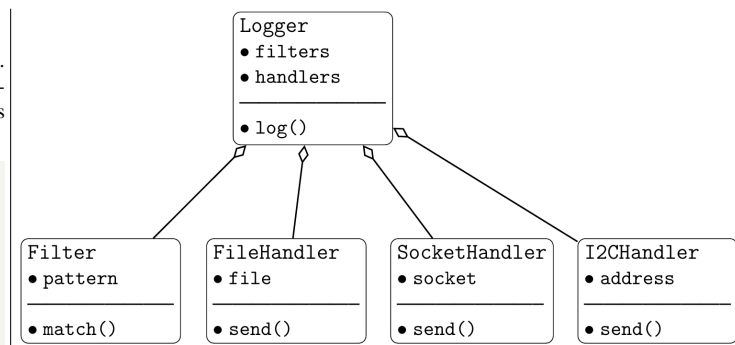
11.2 Aggregation und Komposition

Eine übermäßige Nutzung von Vererbung kann zu dem Problem einer “class explosion” führen und erhöht die Komplexität erheblich, wenn mehrere Traits oder Optionen kombiniert werden.



Aggregation und Komposition bieten strukturelle Alternativen, indem **Klassen basierend auf Abhängigkeiten statt auf strikten Hierarchien zusammengesetzt werden.**

- **Komposition (is-a-part-of):** Bedeutet eine starke Lebenszyklus-Abhängigkeit. Das übergeordnete Objekt besitzt und kontrolliert die Lebensdauer der enthaltenen Objekte. Wenn das übergeordnete Objekt zerstört wird, werden auch seine Teile zerstört (z. B. ein Auto und sein Motor).
- **Aggregation (has-a):** Bedeutet eine lose Lebenszyklus-Abhängigkeit. Enthaltene Objekte können unabhängig vom übergeordneten Objekt existieren und können dynamisch hinzugefügt oder entfernt werden (z. B. eine Bibliothek und ihre Bücher).



```

1 class Filter:
2     def __init__(self, pattern):
3         self.pattern = pattern
4
5 class Logger:
6     # Aggregation: Logger "has" filters and handlers.
7     # The components can exist independently of the Logger
8     # instance.
9     def __init__(self, filters, handlers):
10        self.filters: list[Filter] = filters
11        self.handlers: list[Handlers] = handlers

```

12 NumPy

12.1 Einführung

NumPy (Numerical Python) ist eine Bibliothek zur Durchführung mathematischer Funktionen und Operationen auf großen mehrdimensionalen Arrays. Sie bietet:

- **Geschwindigkeit:** Verwendet optimierten C-Code und ersetzt langsame Standard-Python for-Schleifen.
- **Vektorisierung:** Arraybasierte Operationen werden automatisch elementweise ausgeführt.
- **Broadcasting:** Erweitert implizit Arrays unterschiedlicher Formate während arithmetischen Berechnungen.

12.2 Erstellung von n-dimensionalen Arrays (ndarray)

Das `ndarray` ist eine homogene Datenstruktur, in der jede Dimension als *Achse* bezeichnet wird.

- `arr.ndim`: Anzahl der Dimensionen.
- `arr.shape`: Tupel, das die Größe jeder Dimension darstellt. (rows, cols)
- `arr.dtype`: Datentyp der Array-Elemente (z.B. `np.float64`, `np.int32`).
- `arr.astype(dtype)`: Gibt eine Kopie des Arrays zurück, die in einen bestimmten Typ umgewandelt wurde.

12.2.1 Array-Erstellungsmethoden

```

1 arr = np.array([1, 2, 3]) # Basic 1D array
2 np.ones((3, 3))          # 3x3 array of 1s
3 np.zeros_like(arr)       # Array of 0s with the shape of 'arr'
4 np.full((3, 2), 10)      # 3x2 array filled with 10
5 np.eye(5)                # 5x5 Identity matrix
6 np.linspace(0, 1, 10)    # 10 evenly spaced values from 0 to 1
7 np.arange(0, 4, 0.5)     # Values from 0 to 4 (exclusive),
8                           # step 0.5
9 np.random.randn(3, 2)    # Standard normally distributed
10                          # random numbers

```

```
9 np.random.randint(0, 100, (3)) # 1D arr with 3 Uniformly
distributed integers between 0-100
```

Funktion	Resultat
np.ones((2,2))	array([[1., 1.], [1., 1.]])
np.ones_like(arr1)	array([1, 1, 1])
np.zeros((2,2))	array([[0., 0.], [0., 0.]])
np.zeros_like(arr1)	array([0, 0, 0])
np.full((2,2), 7.0)	array([[7., 7.], [7., 7.]])
np.full_like(arr1, 7)	array([7, 7, 7])
np.eye(2)	array([[1., 0.], [0., 1.]])
np.linspace(0, 1, 5)	array([0., 0.25, 0.5, 0.75, 1.])
np.arange(0, 4, 0.5)	array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5])
np.logspace(0, 1, 4)	array([1., 2.1544, 4.6416, 10.])
np.random.randn(3)	array([0.7576, 0.0135, -0.8934])
np.random.randint(0,10,3)	array([0, 5, 4])

Bei arange ist es möglich, einen negativen step anzugeben, um ein absteigendes Array zu erzeugen.

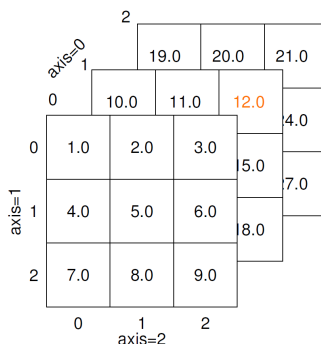
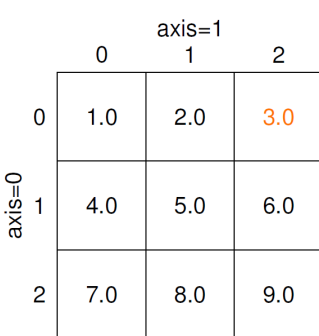
Achten Sie darauf, dass end aus dem Array ausgeschlossen ist, wenn Sie einschließen möchten: (end ± step)

12.3 Elementzugriff

- **Indexing:** Achsen werden durch Kommas innerhalb von eckigen Klammern getrennt (z.B. arr[row, col]).
- **Slicing:** Folgt der [start:end:step]-Syntax über mehrere Achsen hinweg. Slicing gibt eine **view** zurück; Änderungen wirken sich auf das ursprüngliche Array aus. Verwenden Sie .copy(), um eine unabhängige Duplikatkopie zu erstellen. **END not included!**
- **Boolean Slicing:** Extrahiert Teilmengen, bei denen eine Bedingung True ist (z.B. arr[arr > 5]).
- **Unpacking:** Eine Matrix kann direkt in ihre Bestandteile entlang der ersten Achse entpackt werden: row1, row2 = np_matrix

Beispiele:

arr	Ausdruck	Shape	Resultat
	arr[:2, 1:]	(2, 2)	array([[2, 3], [5, 6]])
	arr[2]	(3,)	array([7, 8, 9])
	arr[2, :]	(3,)	array([7, 8, 9])
	arr[2:, :]	(1, 3)	array([[7, 8, 9]])
	arr[:, 2]	(3,)	array([3, 6, 9])
	arr[:, 2:]	(3, 1)	array([[3], [6], [9]])
	arr[1, :2]	(2,)	array([4, 5])
	arr[1:2, :2]	(1, 2)	array([[4, 5]])



Die Achsen ändern sich immer, wenn Dimensionen hinzugefügt/entfernt werden

12.4 Broadcasting

Um Operationen auf Arrays unterschiedlicher Formen auszuführen, broadcastet NumPy automatisch das kleinere Array:

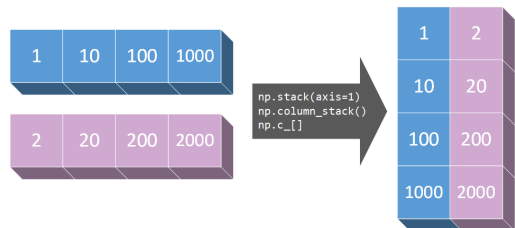
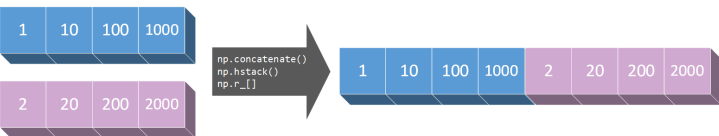
1. Dimensionen werden links eingefügt, bis die Anzahl der Dimensionen (ndim) übereinstimmt.
2. Dimensionen der Größe 1 werden gestreckt, um der entsprechenden Dimension des anderen Arrays zu entsprechen.
3. Wenn die Größen unterschiedlich sind und weder das eine noch das andere 1 ist, wird ein ValueError ausgelöst.

12.5 Array-Manipulation

12.5.1 Verkettung und Stacking

```
1 np.concatenate([a, b], axis=0) # Join along an existing axis
2 np.stack([a, b], axis=0)      # Join along a new axis
3 np.vstack([a, b])             # Stack vertically (row-wise)
4 np.hstack([a, b])             # Stack horizontally (col-wise)
```

```
6 # Slice object concatenation (MATLAB-like)
7 np.r_[1:5, 15, 20:50:10]      # Translates slices to
concatenated 1D array
8 np.c_[0:10]                  # Translates slices to
concatenated 2D columns
```



12.5.2 Dimensionsänderungen

```
1 arr.reshape(2, -1)           # View with new shape (-1 infers
dimension)
2 arr.resize(2, 5)              # In-place shape modification
3 arr.flatten()                 # Returns a 1D COPY
4 arr.ravel()                   # Returns a 1D VIEW
5 arr[np.newaxis, :]           # Increases dimensions
6 np.tile(arr, (5, 2))          # Repeats the entire array for the
given size
7 np.repeat(arr, 3)             # Repeats individual elements
```

- **View vs Copy:** arr.reshape(-1) erzeugt eine **view** und keine physische Kopie. Das Ändern des umgeformten Arrays wirkt sich auf das ursprüngliche aus.
- **Automatic Reshaping:** Die Verwendung von -1 ermöglicht es NumPy, eine Dimension automatisch anhand der gesamten Größe des Arrays zu berechnen.
- **Kompatibilitätsbedingung:** Es kann nur eine Dimension auf -1 gesetzt werden. Damit ein reshape gültig ist, muss die gesamte Anzahl der Elemente N genau durch das Produkt der angegebenen Dimensionen teilbar sein.

12.6 Mathematische Funktionen

NumPy stellt vektorisierte universelle Funktionen (ufuncs) wie np.exp(), np.sin(), np.abs(), np.mean(), np.max(), np.std() und np.cumsum() bereit.

12.7 Verkettung

MATLAB-like Concatenation:

- np.r_: Verkettet entlang der ersten Achse (zeilenweise). Es unterstützt Slice-Objekte und einzelne Werte.
- **Beispiel:** x = np.r_[1:5, 15, 20:50:10] → [1, 2, 3, 4, 15, 20, 30, 40]
- np.c_: Verkettet entlang der zweiten Achse (spaltenweise). Ideal zum Erstellen von Spaltenvektoren.
- **Beispiel:** y = np.c_[0:3] → [[0], [1], [2]]

12.8 NumPy Lineare Algebra: Multiplikation Zusammenfas-

12.8.1 Skalar- und Vektorprodukte

- np.inner(a, b): Am besten für das **Skalarprodukt** (inner product) zweier Vektoren.

- *Wann verwenden:* Verwenden Sie es, wenn Sie explizit das Skalarprodukt von Vektoren möchten. Es ist klarer als `dot()`, weil es auf diese spezifische Operation beschränkt ist.
- *Beispiel:* $v_1 = [1, 2]$; $v_2 = [3, 4] \rightarrow 1 \cdot 3 + 2 \cdot 4 = 11$.
- **np.outer(a, b):** Berechnet das **dyadische Produkt** (outer product).
 - *Wann verwenden:* Verwenden Sie es, um eine Matrix aus zwei Vektoren zu erzeugen (Spaltenvektor $\times$ Zeilenvektor).
 - *Beispiel:* $v_1 = [1, 2]$; $v_2 = [3, 4] \rightarrow [[3, 4], [6, 8]]$.

12.8.2 Matrixmultiplikation: @ vs np.dot()

Während beide die standardmäßige Matrixmultiplikation für 2D-Arrays ausführen, verhalten sie sich für höherdimensionale Tensoren ($n > 2$) unterschiedlich.

- **Der @-Operator (np.matmul):** Der Standard für **Matrixmultiplikation**.
 - *Verhalten* ($n > 2$): Er behandelt das Array als eine 'stack' von Matrizen und multipliziert sie über ihre letzten zwei Dimensionen hinweg.
 - *Empfehlung:* Dies ist die bevorzugte Methode für die standardmäßige Matrixmultiplikation in modernem Python (3.5+).
- **np.dot(a, b):**
 - *Verhalten* ($n > 2$): Es berechnet die Summe der Produkte über der **letzten** Dimension von a und der **vorletzten** Dimension von b .
 - *Ergebnis:* Es multipliziert effektiv die erste Zeile von a mit **JEDER** Spalte in jeder nachfolgenden Matrix von b , was zu einer deutlich größeren resultierenden Form führt.

Meistens immer den @-Operator verwenden

12.8.3 Vergleichsbeispiel ($n > 2$)

```
1 a = np.ones((2, 3, 3))
2 b = np.ones((2, 3, 3))
3
4 # Result: (2, 3, 3) -> Multiplies stacks independently
5 res_matmul = a @ b
6
7 # Result: (2, 3, 2, 3) -> Multiplies every row by every column
8 res_dot = np.dot(a, b)
```

12.8.4 Methoden

Das Modul `np.linalg` stellt essentielle Werkzeuge für Matrixberechnungen bereit.

- **np.linalg.solve(A, b):** Berechnet die exakte Lösung einer linearen Matrixgleichung $Ax = b$.
 - A muss quadratisch und von vollem Rang (nicht singular) sein.
- **np.linalg.lstsq(A, b, rcond=None):** Gibt die **Least-Squares**-Lösung zu einer linearen Matrixgleichung zurück.
 - *Wann verwenden:* Wenn $Ax = b$ keine exakte Lösung hat
 - *Gibt Array zurück:* Ein Tupel, das die Lösung x , die Residuen, den Rang von A und die singulären Werte enthält.
- **np.linalg.eig(A):** Berechnet die Eigenwerte und die rechten Eigenvektoren eines quadratischen Arrays.
 - *Rückgabe:* Ein Tupel (w , v), wobei w die Eigenwerte enthält und v die entsprechende Matrix mit Eigenvektoren als Spalten.
- **np.linalg.inv(A):** Berechnet die multiplikative Inverse einer quadratischen Matrix.
- **np.linalg.det(A):** Berechnet die Determinante einer Matrix.
- **np.trace(A):** Berechnet die Spur, d.h. die Summe der Diagonalelemente eines Arrays.
- **np.linalg.matrix_rank(A):** Gibt den Rang der Matrix eines Arrays zurück.
- **np.linalg.norm(R):** Berechnet die euklidische Norm (Betrag), definiert als die Quadratwurzel aus der Summe der Quadrate der Komponenten des Vektors Formel: $\|p\|_2 = \sqrt{x^2 + y^2 + z^2}$.

Beispielverwendung:

```
1 A = np.array([[3, 1], [1, 2]])
2 b = np.array([9, 8])
3
4 # Exact solution
5 x = np.linalg.solve(A, b)
6
7 # Eigen-decomposition
8 eigenvalues, eigenvectors = np.linalg.eig(A)
9
10 # Least squares (e.g., for linear regression)
11 # rcond=None uses the machine precision default
12 x_ls, res, rank, s = np.linalg.lstsq(A, b, rcond=None)
```

13 Datenvorverarbeitung und -bereinigung

13.1 Resampling

```
1 # --- DOWNSAMPLING ---
2 # 1. Slicing: Take every n-th sample (Fast, but loses
   information)
3 t_down = t[::10]
4
5 # 2. Block Averaging / Mean Pooling: Averages blocks of size n
   (Low-pass filter)
6 s_mean = np.mean(s[:end].reshape(-1, 10), axis=-1)
7
8 # 3. Smoothing and Decimation: Moving average followed by
   slicing
9 kernel = np.ones(20) / 20
10 s_smooth = np.convolve(s, kernel, mode='same')
11 s_down = s_smooth[::10]
12
13 # --- UPSAMPLING ---
14 # 1. Zero-Order Hold (ZOH): Repeats samples
15 s_zoh = np.repeat(s, 4)
16
17 # 2. Interpolation: Estimates missing values (Linear or Cubic)
18 lin_interp = interp1d(t, s, kind='linear', fill_value="
   extrapolate")
19 s_lin = lin_interp(t_up)
```

13.2 Datenbereinigung & Skalierung

```
1 # --- HANDLING NaNs ---
2 nan_mask = np.isnan(data)
3 data_no_nan = data[~nan_mask] # Remove NaNs
4 mean_val = np.nanmean(data) # Compute mean ignoring NaNs
5
6 data_filled = data.copy()
7 data_filled[nan_mask] = mean_val # Replace NaNs with mean
8 data_interp = pd.Series(data).interpolate() # Interpolate missing values
9
10 # --- OUTLIERS & CLIPPING ---
11 clean_data = data[data < 20] # Keep values below threshold
```

```
12 clipped_data = np.clip(data, 0, 10) # Force
   values into [0, 10] range
13
14 # --- FEATURE SCALING ---
15 # 1. Min-Max Normalization (Rescales to [0, 1])
16 normalized = (data - np.min(data)) / (np.max(data) - np.min(
   data))
17
18 # 2. Z-score Standardization (Mean = 0, Std = 1)
19 standardized = (data - np.mean(data)) / np.std(data)
```

13.3 NumPy Dateiverwaltung

```
1 # Save array to a text file with specific formatting
2 x = np.arange(100).reshape(20,5)
3 np.savetxt("my_data.txt", x, fmt="%2d", delimiter=",")
4
5 # Load specific columns from a text file
6 y = np.loadtxt("my_data.txt", delimiter=",", usecols=(0, 2, 4)
   )
```

14 Data-visualization

Datenvisualisierung ist entscheidend, weil Kennzahlen der Zusammenfassung irreführend sein können und die Wahl des Diagramms die gezogenen Schlussfolgerungen direkt beeinflusst. Matplotlib ist eine umfassende Bibliothek zur Erstellung statischer, animierter und interaktiver Visualisierungen in Python.

14.1 Matplotlib APIs

Es gibt im Wesentlichen zwei Ansätze zur Erstellung von Diagrammen: den nicht-objektorientierten Ansatz über die funktionale Schnittstelle (pyplot) und den objektorientierten (OOP) Ansatz. Der OOP-Ansatz wird empfohlen, da er mehr Kontrolle und Anpassungsmöglichkeiten bietet.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 t = np.linspace(0, 1, 100)
5 s1 = np.sin(2 * np.pi * t)
6
7 # Object-Oriented API (Recommended)
8 fig, ax = plt.subplots(layout="constrained")
9 ax.plot(t, s1, linestyle='-', marker='.', color="blue", label=
   'Simulation')
10 ax.set_xlabel('Time (s)')
11 ax.set_ylabel('Amplitude (V)')
12 ax.grid(which="both")
13 ax.legend()
14 plt.show()
```

14.2 Häufige Methoden und Formatierungsargumente

Beim Anpassen von Diagrammen werden mehrere zentrale Argumente häufig über verschiedene Matplotlib-Methoden hinweg verwendet:

- **Abbildung und Layout (plt.subplots, plt.figure):**
 - `figsize=(width, height)`: Legt die Abmessungen der Figure in Zoll fest.
 - `layout="constrained"`: Passt Subplots automatisch so an, dass sie in den Bereich der Figure passen, und verhindert überlappende Elemente.
 - `sharex, sharey`: Erzwingt, dass Subplots dieselbe Skalierung der X- oder Y-Achse teilen (nützlich für direkte Vergleiche).

- **Plot-Styling (ax.plot, ax.scatter):**
 - color oder c: Legt die Farbe der Diagrammelemente fest.
 - linestyle oder ls: Definiert den Liniensstil ('-' durchgezogen, '-' gestrichelt, '-.' strichpunktierter, ':' gepunktet).
 - linewidth oder lw: Legt die Strichstärke der Linie fest.
 - marker: Fügt den Datenpunkten spezifische Symbole hinzu.
 - alpha: Steuert die Transparenz (0.0 ist vollständig transparent, 1.0 ist vollständig deckend).
 - label: Weist dem Plot-Trace eine Zeichenkette als Bezeichner zu, die anschließend von ax.legend() verwendet wird.
- **Text und Beschriftungen (ax.set_title, ax.set_xlabel, ax.text):**
 - fontsize: Gibt die Größe der Textelemente an.
 - ha, va: Horizontale und vertikale Ausrichtung für Textannotationen (z.B. 'center', 'left', 'top', 'bottom').
 - loc: Positionsargument, das üblicherweise zur Platzierung von Legenden verwendet wird (z.B. ax.legend(loc='upper right')).

14.3 Gängige Plot-Typen

Je nach Daten sind unterschiedliche Diagrammtypen effektiver:

- **Line Plots:** Nützlich für kontinuierliche Signale. Logarithmische Achsen können mit loglog(), semilogx() oder semilogy() angewendet werden.
- **Step Plots:** Werden mit step() erstellt und zeichnen Daten mit einer horizontalen Basislinie, die durch vertikale Linien verbunden ist.
- **Histograms:** Visualisieren die numerische Verteilung von Daten mit hist().
- **Scatter Plots:** Stellen Messdaten als Punkte auf den X- und Y-Achsen mit scatter() dar. Größe und Farbe können dritte und vierte Dimensionen repräsentieren.
- **3D and Contour Plots:** contour() oder contourf() für 2D-Querschnitte einer 3D-Oberfläche und plot_surface() für 3D-Renderings.

14.4 Subplots und Layouts

Beim Verwalten mehrerer Diagramme erstellt plt.subplots() eine Figure und ein Array von Achsenobjekten.

```
1 # Erstellen eines 2x2-Rasters von Subplots mit gemeinsamen X-
  und Y-Achsen
2 fig, axes = plt.subplots(nrows=2, ncols=2, sharex=True, sharey
  =True)
3 axes[0, 0].plot(t, s1)
4 axes[0, 0].set_title("First Plot")
```

14.5 Bibliotheksvergleich

- **Matplotlib:** Vollständige Kontrolle und Anpassung, sehr anpassbar, aber statisch.
- **Seaborn:** Auf Matplotlib aufbauend, bietet High-Level-statistische Diagramme mit klaren Standardstilen und integriert sich perfekt mit Pandas.
- **Plotly:** Webbasierte Diagramme, die Zoomen, Hovering und dynamische Aktualisierungen ermöglichen.

15 Anhang

15.1 Typumwandlung: dict() und list()

- **Tuple zu Dict:** Die Funktion dict() kann eine Liste von 2-Element-Tupeln direkt in Schlüssel-Wert-Paare umwandeln.
- **Duplikatentfernung:** Da Wörterbuchschlüssel eindeutig sein müssen, entfernt das Umwandeln einer Tupelliste in ein dict automatisch doppelte Einträge (wobei der zuletzt ausgewertete Wert für einen gegebenen Schlüssel beibehalten wird).
- **Dict zu List:** Wandelt das deduplierte Wörterbuch zurück in eine Liste von Tupeln, indem list(dictionary.items()) verwendet wird.

```
1 # List of tuples containing duplicate entries
```

```
2 contacts = [
3     ('Maria', '022 548 95 26'),
4     ('Patrick', '081 445 66 58'),
5     ('Patrick', '081 445 66 58') # Duplicate
6 ]
7
8 # 1. dict(contacts) removes the duplicate 'Patrick' key
9 # 2. .items() returns the key-value pairs
10 # 3. list() converts them back into a list of tuples
11 data_unique = list(dict(contacts).items())
12
13 print(data_unique)
14 # Output: [('Maria', '022 548 95 26'), ('Patrick', '081 445 66
  58')]
```

15.2 Decorators

Decorators sind Funktionen, die eine andere Funktion als Argument annehmen, um ihr Verhalten zu erweitern, ohne den ursprünglichen Quellcode zu verändern. Sie werden durch das Symbol @ gekennzeichnet.

```
1 def require_login(func):
2     def wrapper(user):
3         if user["logged_in"]:
4             return func(user)
5         print("Access denied.")
6     return wrapper
7
8 @require_login
9 def view_dashboard(user):
10    print(f"Welcome {user['name']}")
```

15.3 Skriptausführung

Um zu unterscheiden, ob ein Script direkt ausgeführt wird oder als Modul importiert wird, verwendet Python die Variable __name__.

- Wenn direkt ausgeführt: __name__ == "__main__".
- Wenn importiert: __name__ entspricht dem Dateinamen des Moduls.

```
1 def main():
2     # Test code or entry point
3     pass
4
5 if __name__ == "__main__":
6     main()
```

15.4 Zufallsverteilungen in NumPy

- **np.random.rand:** Erzeugt Stichproben aus einer **Gleichverteilung** im Intervall [0, 1). Jeder Wert hat die gleiche Wahrscheinlichkeit, gezogen zu werden.
- **np.random.randn:** Erzeugt Stichproben aus der **Standardnormalverteilung** (Gaußverteilung) mit dem Mittelwert $\mu = 0$ und der Varianz $\sigma^2 = 1$.

15.4.1 Transformation der Normalverteilung

Um die Standardnormalverteilung $N(0, 1)$ zu einer benutzerdefinierten Verteilung $N(\mu, \sigma^2)$ zu skalieren und zu verschieben, wird die folgende lineare Formel angewendet:

$$X = \sigma \cdot \text{np.random.randn}(\dots) + \mu$$

Wobei:

- σ (Sigma): Standardabweichung (prüft die "Breite" der Glocke).
- μ (Mu): Erwartungswert (verschiebt das Zentrum der Verteilung).

15.5 Indexing vs. Slicing: Faustregel

Der Unterschied hängt vom Typ des Selektors ab, der für die Achse verwendet wird:

- **Ganzzahl (Indexing):** Wenn du eine einzelne Zahl verwendest (z.B. arr[2]), wird die Dimension **entfernt** (rank-1).
- **Bereich (Slicing):** Wenn du die beiden Doppelpunkte verwendest (z.B. arr[2:3]), wird die Dimension **beibehalten** (rank-2).

15.5.1 Praxisbeispiel

Gegeben sei ein 2D-Array mit Shape (3, 3):

- arr[2, :] → Ergebnis 1D, Shape (3,).
- arr[2:3, :] → Ergebnis 2D, Shape (1, 3).

Python – Summary

Built-in Functions

abs(x) ⇒ num

all(iterable) ⇒ bool

any(iterable) ⇒ bool

ascii(object) ⇒ str

bin(x) ⇒ str

bool([x]) ⇒ bool

breakpoint(*args, **kws)

bytearray([source[,encoding[,errors]]]) ⇒ bytearray

bytes([source[, encoding[, errors]]]) ⇒ bytes

callable(object) ⇒ bool

chr(i) ⇒ str

@classmethod

compile*

complex([real[, imag]]) ⇒ complex

delattr(object, name)

dict(**kwargs) ⇒ dict

dict(mapping, **kwargs) ⇒ dict

dict(iterable, **kwargs) ⇒ dict

dir([object]) ⇒ list

divmod(a, b) ⇒ num

enumerate(iterable, start=0) ⇒ enumerate object

eval(expression[, globals[, locals]]) ⇒ object

exec(object[, globals[, locals]])

filter(function, iterable)

float([x]) ⇒ float

format(value[, format_spec]) ⇒ str

frozenset([iterable]) ⇒ frozenset

getattr(object, name[, default]) ⇒ object

globals() ⇒ dict

hasattr(object, name) ⇒ bool

hash(object) ⇒ int

help([object])

hex(x) ⇒ str

id(object) ⇒ int

input([prompt]) ⇒ str

int([x]) ⇒ class

int(x, base=10) ⇒ int

isinstance(object, classinfo) ⇒ bool

issubclass(class, classinfo) ⇒ bool

*

__import__(name, globals=None, locals=None, fromlist=(), level=0)

compile(source, filename, mode, flags=0, dont_inherit=False, optimize=-1) ⇒ AST object

print(*objects, sep=, end="\\n", file=sys.stdout, flush=False)

property(fget=None, fset=None, fdel=None, doc=None) ⇒ property attribute

str(object=b, encoding="utf-8", errors="strict") ⇒ str

open(file, mode="r", buffering=-1, encoding=None, errors=None, newline=None, ...) ⇒ file object (file)

mode = "r" | "w" | "x" | "a" | "b" | "t" | "+"

file.close()

file.read(size) ⇒ str | bytes

file.readlines(hint) ⇒ list

file.writelines(list)

file.readline(size) ⇒ str

file.write(str | bytes)

iter(object[, sentinel]) ⇒ iterator

len(s) ⇒ int

list([iterable]) ⇒ list

locals() ⇒ dict

map(function, iterable, ...) ⇒ iterator

max(iterable, *[, key, default]) ⇒ object

max(arg1, arg2, *args[, key]) ⇒ object

min(iterable, *[, key, default]) ⇒ object

min(arg1, arg2, *args[, key]) ⇒ object

next(iterator[, default]) ⇒ object

object ⇒ object

oct(x) ⇒ str

open*

ord(c) ⇒ int

pow(base, exp[, mod]) ⇒ num

print*

property*

range(stop) ⇒ sequence object

range(start, stop[, step]) ⇒ sequence object

repr(object) ⇒ str

reversed(seq) ⇒ iterator

round(number[, ndigits]) ⇒ num

set([iterable]) ⇒ set

setattr(object, name, value)

slice(stop) ⇒ slice object

slice(start, stop[, step]) ⇒ slice object

sorted(iterable, *, key=None, reverse=False) ⇒ list

@staticmethod

str(object=) ⇒ str

str*

sum(iterable, /, start=0) ⇒ num

super([type[, object-or-type]]) ⇒ proxy object

tuple([iterable]) ⇒ tup

type(object) ⇒ type object

type(name, bases, dict) ⇒ type

vars([object]) ⇒ ...dict... attribute

zip(*iterables) ⇒ iterator

__import__*

Dict Methods

dict.clear()

dict.copy() ⇒ dict

dict.fromkeys(iterable[, value]) ⇒ dict

dict.get(key[, default]) ⇒ object

dict.items() ⇒ dict.items

dict.keys() ⇒ dict.keys

dict.pop(key[, default]) ⇒ object

dict.popitem() ⇒ tup

dict.setdefault(key[, default]) ⇒ object

dict.update([other])

dict.values() ⇒ dict.values

List Methods

list.append(x)

list.clear()

list.copy() ⇒ list

list.count(x) ⇒ int

list.extend(iterable)

list.index(x[, start[, end]]) ⇒ int

list.insert(i, x)

list.pop([i]) ⇒ object

list.remove(x)

list.reverse()

list.sort(key=None, reverse=False)

String Methods

Testing

str.endswith(suffix[, start[, end]]) ⇒ bool

str.isalnum() ⇒ bool

str.isalpha() ⇒ bool

str.isascii() ⇒ bool

str.isdecimal() ⇒ bool

str.startswith(prefix[, start[, end]]) ⇒ bool

str.isdigit() ⇒ bool

str.isidentifier() ⇒ bool

str.islower() ⇒ bool

str.isnumeric() ⇒ bool

str.isspace() ⇒ bool

str.istitle() ⇒ bool

str.isupper() ⇒ bool

str.isprintable() ⇒ bool

Formatting

str.center(width[, fillchar]) ⇒ str

str.format(*args, **kwargs) ⇒ str

str.format_map(mapping) ⇒ str

str.rjust(width[, fillchar]) ⇒ str

str.ljust(width[, fillchar]) ⇒ str

str.zfill(width) ⇒ str

Transformation

str.encode*

str.expandtabs(tabsize=8) ⇒ str

str.lstrip([chars]) ⇒ str

str.replace(old, new[, count]) ⇒ str

str.rstrip([chars]) ⇒ str

str.strip([chars]) ⇒ str

*

str.encode(encoding="utf-8", errors="strict") ⇒ bytes object

Upper and Lower Cases

str.capitalize() ⇒ str

str.casefold() ⇒ str

str.lower() ⇒ str

str.swapcase() ⇒ str

str.title() ⇒ str

str.upper() ⇒ str

Split & Combine

str.join(iterable) ⇒ str

str.partition(sep) ⇒ tup

str.rpartition(sep) ⇒ tup

str.split(sep=None, maxsplit=-1) ⇒ list

str.rsplit(sep=None, maxsplit=-1) ⇒ list

str.splitlines([keepends]) ⇒ list

Searching

str.count(sub[, start[, end]]) ⇒ int

str.find(sub[, start[, end]]) ⇒ int

str.index(sub[, start[, end]]) ⇒ int

str.rfind(sub[, start[, end]]) ⇒ int

str.rindex(sub[, start[, end]]) ⇒ int

String Formatting {<f_expression> ["="] [!<conversion>] [:<format_spec>]}

conversion s: with str(), r: with repr(), a: with ascii(),

format_spec [[fill]align][sign][#][0][width][group][.precision][type]

fill

align

sign

width

group

precision

type

<any character> (except "{" and "}")

"<" | ">" | "=" | "^"

"+" | "- " | " "

positive integer

" " | ".,"

positive integer

"b" | "c" | "d" | "e" | "E" | "f" | "F" | "g" | "G" | "n" | "o" | "s" | "x" | "X" | "%"

Set Methods

set.add(elem)

set.clear()

set.copy() ⇒ set

set.difference(*others) ⇒ set

set.difference_update(*others)

set.discard(elem)

set.intersection(*others) ⇒ set

set.intersection_update(*others)

set.isdisjoint(bool)

set.issubset(other) ⇒ bool

set.issuperset(other) ⇒ bool

set.pop() ⇒ object

set.remove(elem)

set.symmetric_difference(other) ⇒ set

set.symmetric_difference_update(other)

set.union(*others) ⇒ set

set.update(*others)

NumPy (import numpy as np) **Legende:** "⇒ndarray" = copy | "⇒ndarray◊" = view | "⇒ndarray(◊)" = view/copy

Array Creation

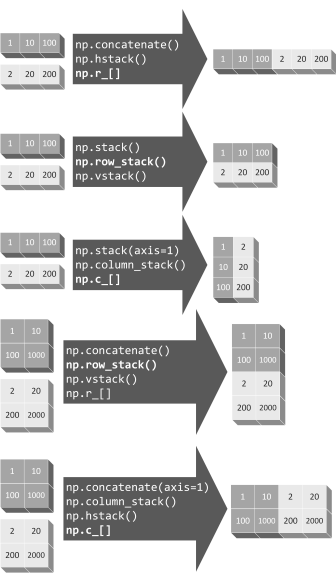
np.array(object, dtype=None, *, copy=True, order="K", subok=False, ndmin=0)⇒ndarray
np.arange([start,] stop[, step,]dtype=None)⇒ndarray
np.asarray(a[, dtype, order])⇒ndarray(◊)
np.copy(a, order="K", subok=False)⇒ndarray
np.empty(shape, dtype=float, order="C")⇒int⇒ndarray
np.empty_like(prototype, dtype=None, order="K", subok=True, shape=None)⇒ndarray
np.eye(N, M=None, k=0, dtype=<class "float">, order="C")⇒ndarray
np.full(shape, fill_value, dtype=None, order="C")⇒ndarray
np.full_like(a, fill_value, dtype=None, order="K", subok=True, shape=None)⇒ndarray
np.loadtxt(file, dtype=float, count=-1, sep=, offset=0)⇒ndarray
np.fromstring(string, dtype=float, count=-1, sep=)⇒ndarray
np.identity(n, dtype=None)⇒ndarray
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)⇒ndarray
np.loadtxt(fname, dtype=<class "float">, comments="#", delimiter=None, converters=None, skiprows=0, usecols=None, unpack=False, ndmin=0, encoding="bytes", max_rows=None)⇒ndarray
np.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)⇒ndarray
np.meshgrid(*xi, copy=True, sparse=False, indexing='y')⇒ndarray
np.ones(shape, dtype=None, order="C")⇒ndarray
np.ones_like(a, dtype=None, order="K", subok=True, shape=None)⇒ndarray
np.random.rand(d0, d1, ..., dn)⇒ndarray
np.random.randn(d0, d1, ...,dn)⇒ndarray
np.random.randint(low, high=None, size=None, dtype=int)⇒ndarray
np.zeros(shape, dtype=float, order="C")⇒ndarray
np.zeros_like(a, dtype=None, order="K", subok=True, shape=None)⇒ndarray

Array Manipulation

numpy.append(arr, values, axis=None)⇒ndarray
np.array_split(ary, indices_or_sections, axis=0)⇒list
np.atleast_1d(*args)⇒ndarray
np.atleast_2d(*args)⇒ndarray
np.atleast_3d(*args)⇒ndarray
np.c[...]⇒ndarray
np.column_stack(tup)⇒ndarray
np.concatenate((a1, a2, ...), axis=0, out=None)⇒ndarray
np.dstack(tup)⇒ndarray
np.expand_dims(a, axis)⇒ndarray◊
np.ndarray.flatten([order])⇒ndarray
np.hstack(tup)⇒ndarray
np.r[...]⇒ndarray

np.ravel(a, order="C")⇒ndarray⇒array_like(◊)
np.repeat(a, repeats, axis=None)⇒ndarray
np.reshape(a, newshape, order="C")⇒ndarray(◊)
np.resize(a, newshape)⇒ndarray
np.roll(a, shift, axis=None)⇒ndarray
np.row_stack(tup)⇒ndarray
np.split(ary, indices_or_sections, axis=0)⇒list◊
np.stack(arrays, axis=0, out=None)⇒ndarray
np.tile(A, reps)⇒ndarray
np.transpose(a, axes=None)⇒ndarray
np.ndarray.T⇒ndarray(◊)
np.unravel_index(indices, shape, order="C")⇒tup
np.vstack(tup)⇒ndarray

Description	1D→1D	1D→2D	2D→2D	2D→3D
np.concatenate(arrays, axis=0, out=None) Join a sequence of arrays along an existing axis.	0 ¹	-	0/1	-
np.stack(arrays, axis=0, out=None) Join a sequence of arrays along a new axis.	-	0/1	-	0/1/2
np.column_stack(tup) Stack 1-D arrays as columns into a 2-D array.	-	1	1	-
np.row_stack(tup) Stack arrays in sequence vertically (row wise).	-	0	0	-
np.hstack(tup) Stack arrays in sequence horizontally (col wise).	0	-	1	-
np.vstack(tup) Stack arrays in sequence vertically (row wise).	-	0	0	-
np.c_[] Translates slice objects to concat. along the second axis.	-	1	1	-
np.r_[] Translates slice objects to concat. along the first axis.	0	-	0	-



¹ This number specifies the axis over which the arrays are stacked.

Mathematical Functions

np.arccos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.arcsin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.arctan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.arctan2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.ceil(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray|scalar
np.cos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.cumsum(a, axis=None, dtype=None, out=None)⇒ndarray
np.degrees(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.diff(a, n=1, axis=-1, prepend=<no value>, append=<no value>)⇒ndarray
np.exp(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray|scalar
np.floor(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray|scalar
np.log(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.log10(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.radians(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.sin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)⇒ndarray
np.sqrt(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray
np.tan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])⇒ndarray

Linear Algebra

np.cross(a, b, axisa=-1, axisb=-1, axisc=-1, axis=None)⇒ndarray
np.dot(a, b, out=None)⇒ndarray
np.linalg.det(a)⇒array_like
np.linalg.eig(a)⇒ndarray
np.inner(a, b)⇒ndarray
np.linalg.inv(a)⇒ndarray
np.matmul(x1, x2, **kwargs)⇒ndarray
np.linalg.matrix_rank(M, tol=None, hermitian=False)⇒array_like
np.linalg.norm(x, ord=None, axis=None, keepdims=False)⇒float|ndarray
np.outer(a, b, out=None)⇒ndarray
np.linalg.solve(a, b)⇒ndarray
M @ v⇒ndarray
np.trace(a, offset=0, axis1=0, axis2=1, dtype=None, out=None)⇒ndarray

Matplotlib (import matplotlib.pyplot as plt)

Creation

plt.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True, FigureClass=<class "matplotlib.figure.Figure">, clear=False, **kwargs)⇒Figure
plt.subplot(*args, **kwargs)⇒AxesSubplot
plt.subplots(nrows=1, ncols=1, *, sharex=False, sharey=False, squeeze=True, subplot_kw=None, gridspec_kw=None, **fig_kw)⇒Figure, axes.Axes
plt.twinx(ax=None)⇒AxesSubplot
plt.twiny(ax=None)⇒AxesSubplot
plt.tight_layout(*, pad=1.08, h_pad=None, w_pad=None, rect=None)
plt.savefig(*args, **kwargs)
plt.show(*, block=None)

Drawing

plt.annotate(text, xy, *args, **kwargs)⇒Annotation
plt.arrow(x, y, dx, dy, **kwargs)⇒FancyArrow
plt.contour(*args, data=None, **kwargs)⇒QuadContourSet
plt.contourf(*args, data=None, **kwargs)⇒QuadContourSet
plt.loglog(*args, **kwargs)⇒list of Line2D objects
plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs)⇒list of Line2D objects
plt.scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None, vmin=None, vmax=None, alpha=None, linewidths=None, verts=<deprecated parameter>, edgecolors=None, *, plotnonfinite=False, data=None, **kwargs)⇒PathCollection
plt.semilogx(*args, **kwargs)⇒list of Line2D objects
plt.semilogy(*args, **kwargs)⇒list of Line2D objects
plt.stem(*args, linefmt=None, markerfmt=None, basefmt=None, bottom=0, label=None, use_line_collection=True, data=None)⇒StemContainer
plt.step(x, y, *args, where="pre", data=None, **kwargs)⇒list of Line2D objects
plt.streamplot(x, y, u, v, density=1, linewidth=None, color=None, cmap=None, norm=None, arrowsize=1, arrowstyle="->|>", minlength=0.1, transform=None, zorder=None, start_points=None, maxlength=4.0, integration_direction="both", *, data=None)⇒StreamplotSet
plt.text(x, y, s, fontdict=None, **kwargs)⇒Text

Decoration

```
plt.axis(*args, emit=True, **kwargs) ⇒ Annotation
plt.axhline(y=0, xmin=0, xmax=1, **kwargs) ⇒ Line2D
plt.axvline(x=0, ymin=0, ymax=1, **kwargs) ⇒ Line2D
plt.colorbar(mappable=None, cax=None, ax=None, **kw) ⇒ Colorbar
plt.grid(b=None, which="major", axis="both", **kwargs)
plt.legend(*args, **kwargs)
plt.margins(*margins, x=None, y=None, tight=True) ⇒ tup
plt.suptitle(t, **kwargs) ⇒ Text
plt.title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ⇒ Text
plt.xlabel(xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.xlim(*args, **kwargs) ⇒ tup
plt.xticks(ticks=None, labels=None, **kwargs) ⇒ tup
plt.ylabel(ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.ylim(*args, **kwargs) ⇒ tup
plt.yticks(ticks=None, labels=None, **kwargs) ⇒ tup
```

Decoration OOP (The names of the listed OOP methods differ from the non-OOP method names.)

```
Axes.set_title(self, label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ⇒ Text
Axes.set_xlabel(self, xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_xlim(self, left=None, right=None, emit=True, auto=False, *, xmin=None, xmax=None) ⇒ tup
Axes.set_xticks(self, ticks, *, minor=False) ⇒ tup
Axes.set_ylabel(self, ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_ylim(self, bottom=None, top=None, emit=True, auto=False, *, ymin=None, ymax=None) ⇒ tup
Axes.set_yticks(self, ticks, *, minor=False) ⇒ tup
```

Linestyle	Marker	Color
"-" "solid"	"." ● "o" ● "v" ▼	"b", "blue" "g", "green" "r", "red"
":" "dotted"	"s" ■ "p" ◆ ">" ►	"c", "cyan" "m", "magenta" "k", "black"
"--" "dashed"	"*" ★ "d", "D" ♦ "^" ▲	"w", "white" "y", "yellow" "#0F0F0F"
"-." "dashdot"	"x", "X" × "+", "p" + "<" ◄	(0.0, 0.5, 1.0)